Overview of actions

		Action	Description	Time spent(min)
		Core Setup Actions:		
User Story	"UNet – VGG19 encoder (256×256)" - Objective: Implement a CNN used for Image Segmentation on TT-NN. Start by PyTorch prototyping and move to deployment on Tenstorrent hardware with TT-NN. This developer journey is designed to go over environment setup, dataset preparation, model design, parameter conversion, input preprocessing, TT-NN implementation of UNet components, and validation against the PyTorch baseline. Complete each step individually in a modular fashion if possible to ensure easy verifiability. As a developer you should be able to reproduce the PyTorch baseline, port the model to TT accelerators, and optimize it for accuracy, correctness, and efficiency.	Install deps	pip install torch torchvision ttnn albumentations opencv-python (plus your experiment stack)	2
		Verify installation	Run a tiny smoke test: import libs, query device, simple ttnn tensor ops	2
		Initialize device	Open TT device(s), set mesh, precision (e.g., FP16/BF16), seeds, deterministic flags	4
		Data & Preprocessing Actions:		
		Define dataset interface	Create SegDataset reading images/masks; support train/val/test splits	5
		Image sizing	Center-crop/resize to 256×256 (keep aspect policy documented)	5
As a model developer I want to run an image segmentation model on Tenstorrent Hardware		Augmentation (train)	Albumentations: flips, small rotations, elastic/affine, color jitter (for robustness)	5
		Normalization	Per-channel mean/std (ImageNet if using pretrained VGG19); scale masks to {0,1}	5
		Batching	Implement PyTorch DataLoader with worker count/pinning; ensure mask dtype long/float	5
		Tiling (optional)	For >256 inputs, optional sliding-window/overlap tiling & reassembly pipeline	
		Model Architecture Actions:		
		Define VGG19 encoder	Use torchvision VGG19 (optionally pretrained=True), extract feature blocks at 1/2/4/8/16 scales	
		Freeze policy	Choose which encoder stages to freeze/unfreeze (warm-up, then unfreeze deeper blocks)	
		Bridge layers	1×1 or 3×3 convs to adjust channel dims from VGG blocks to UNet skip widths	
		UNet decoder	5-stage up path: Upsample (transpose conv or bilinear+3×3 conv), concat with skip, double conv	
		Skip connections	Wire encoder feature maps to matching decoder stages; handle size off-by-one with padding/crop	
		Final head	1×1 conv → C_out classes; activation: sigmoid (binary) or softmax (multi-class)	
		Exportable model	Keep clean module boundaries so you can export weights per-layer for TT-NN	
		Training & Baseline (PyTorch):		
		Losses	Binary/multi-class: BCE/BCEWithLogits + Dice; or Focal + Dice (log-cosh Dice optional)	
		Metrics	Dice, IoU (mIoU), Pixel Acc, Precision/Recall; compute per-class if multi-class	
		Optimizer & sched	AdamW/SGD; cosine/plateau sched; mixed precision (torch.cuda.amp) for speed	
		Baseline checkpoint	Train to convergence on 256×256; save state_dict and per-layer shapes for conversion	
		Sanity visualizations	Save overlay grids (input/pred/mask) for a fixed validation set	
		Parameter Conversion Actions (PyTorch → TT-NN):		
		Extract weights	Walk modules in deterministic order; record (name, type, shape, dtype)	
		Conv weight layout	Convert from PyTorch (out, in, kH, kW) to TT-NN expected layout; handle groups=1	
		Bias terms	Export/attach biases per conv/bridge/head; default to zeros if layer had no bias	
		BatchNorm folding	Fold BN into preceding conv (scale/shift using running stats) to simplify TT graph	
		Activation parity	Confirm TT-NN uses same nonlinears (ReLU, LeakyReLU); document in-place vs out-of-place	
		Save conversion map	Emit a JSON manifest mapping PyTorch modules → TT-NN ops, including tensor transposes	
		Load to device	Create TT tensors with correct memory layout/stride; upload to TT device(s)	
		Input Pipeline Actions (TT-NN):		
		Preprocess to TT	Apply the exact normalization as in PyTorch; convert NCHW fp32 → device dtype/layout	
		Batch handling	Support 1 and N>1; document max batch that fits L1/DRAM; pad to tiles if needed	
		I/O staging	Asynchronous host→device copy; pre-allocate input/output buffers for steady-state runs	
		UNet-Specific Implementation Actions (TT-NN):		
		Implement conv blocks	ttnn.conv2d (3×3, stride=1, padding=1) ×2 per stage; verify padding equivalence	
		Downsampling	MaxPool2d stride=2 (or strided conv); ensure exact output sizes vs PyTorch	
		Upsampling	Transposed conv 2× (preferred) or bilinear upsample + 3×3 conv; match PyTorch semantics	
		Concat skips	Channel-wise concat of upsampled map + encoder skip; adjust padding/cropping to match	
		Nonlinearities	ttnn.relu (or chosen activation); ensure no in-place mutations affecting grads (if training)	
		Final logits	1×1 conv to C_out; optional ttnn.sigmoid/softmax done post-hoc for evaluation	
		Validation & Debug Actions:		
		Shape parity checks	Print per-stage shapes (enc/dec) PyTorch vs TT-NN on a fixed input	
		Numerical checks	Layer-by-layer RMS/Max error; tolerate minor fp16/bf16 drift; assert thresholds	
		Whole-model parity	Compare full forward outputs; compute Dice/IoU deltas on a small val batch	
		Edge cases	Non-divisible sizes (if you allow >256), odd kernels, off-by-one crops, empty masks	
		Performance Measurement Actions:		
		Warm-up & timing	Discard warm-ups; measure median/percentiles over many iters (steady state)	
		Throughput & latency	Report imgs/sec and p50/p95 latency; also host↔device transfer time	
		Memory profiling	Peak device DRAM/L1; activation checkpointing options; workspace sizes	